KONGU ENGINEERING COLLEGE

TRANSFORM YOURSELF

Transform Yourself

22CDF03 DEVOPS

INFRASTRUCTURE PROVISIONING USING TERRAFORM

PRESENTED BY :

RUTHUREASWARAN R	23CDR140
SANJAI S	23CDR146
VILVASHINI T	23CDR184

BATCH NO: C01

KONGU

Assuring the best

PROBLEM STATEMENT

- No centralized platform for student note management
- Manual deployments cause downtime and inconsistency
- No automated infrastructure — setup must be repeated manually
- No rollback mechanism if a deployment fails
- Environment mismatches between development and production

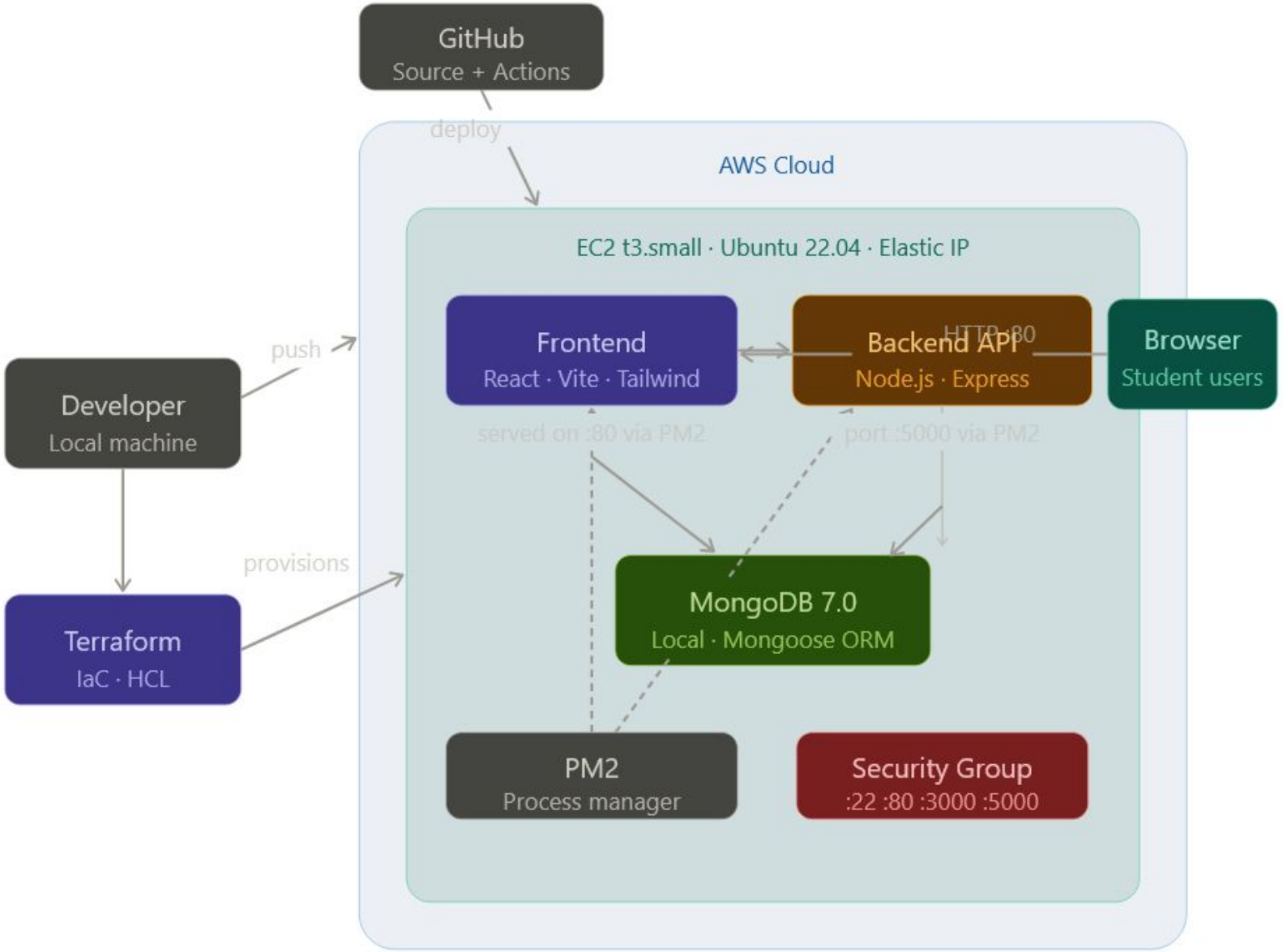
KONGU ENGINEERING COLLEGE

OBJECTIVES

- Automate infrastructure provisioning using Terraform
- Enable zero-touch deployment
- Ensure consistent environments using automated cloud-init scripts
- Eliminate manual AWS Console steps with reproducible, version-controlled IaC
- Use PM2 for process management to keep services alive after deployment

KONGU ENGINEERING COLLEGE

SYSTEM ARCHITECTURE



KONGU ENGINEERING COLLEGE

TOOLS USED

- Terraform – Infrastructure as Code
- AWS EC2 – Cloud compute hosting both frontend and backend
- PM2 – Production process manager to keep Node.js running
- GitHub Actions – CI/CD pipeline — triggers deployment on code push

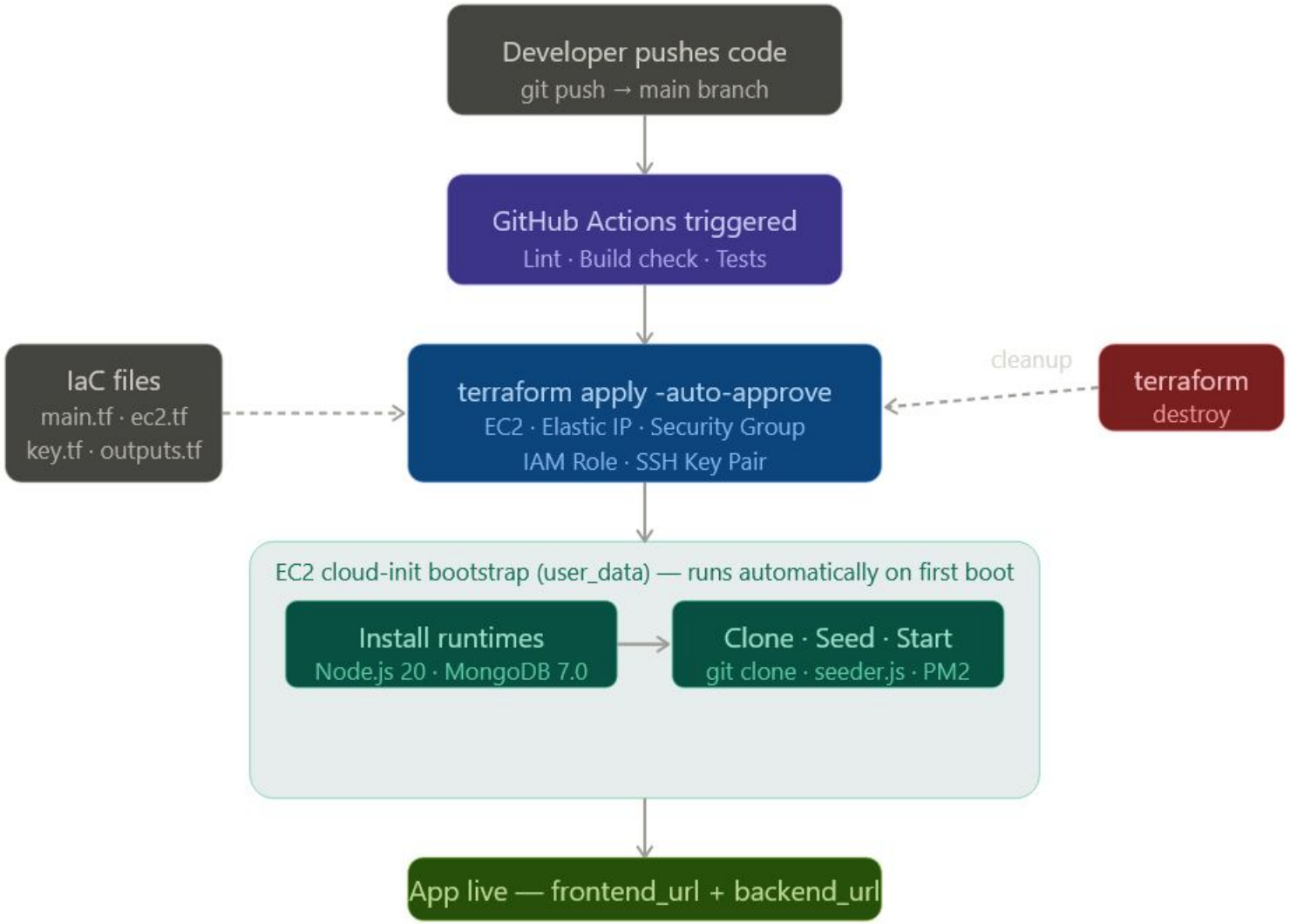
KONGU ENGINEERING COLLEGE

TECH STACK

- **Version Control:**
  - ❖ GitHub
- **CI/CD Pipeline:**
  - ❖ GitHub Actions
- **Cloud:**
  - ❖ Amazon Web Services
- **IaC:**
  - ❖ Terraform
- **Process Manager:**
  - ❖ PM2

KONGU ENGINEERING COLLEGE

WORK FLOW / PIPELINE



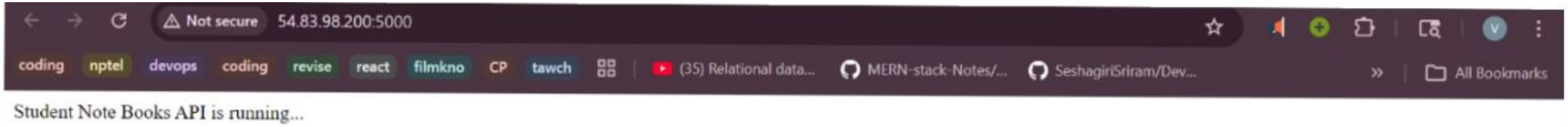
KONGU ENGINEERING COLLEGE

IMPLEMENTATION / COMMANDS

IMPLEMENTATION / COMMANDS		
CATEGORY	KEY COMMAND	BRIEF PURPOSE
Version Control (Git)	git push origin main	Pushes code to GitHub and triggers the CI/CD pipeline.
CI/CD (GitHub Actions)	Auto workflow trigger	Automatically runs build, test, and deploy on every push.
IaC Init (Terraform)	terraform init	Initialises Terraform and downloads the AWS provider plugin.
Plan (Terraform)	terraform plan	Previews infrastructure changes before applying them.
Deploy (Terraform)	terraform apply -auto-approve	Provisions EC2, Elastic IP, Security Group and bootstraps the app.
DB Seed (MongoDB)	node seeder.js	Seeds the MongoDB database with initial data automatically on boot.
Process Manager (PM2)	pm2 start server.js --name api	Starts the Node.js backend and keeps it alive in production.
Process Status (PM2)	pm2 status	Lists all running processes and their current status.
Frontend Build (AWS)	npm run build && serve -s dist	Builds the React app and serves it on port 80 on EC2.
Teardown (Terraform)	terraform destroy -auto-approve	Destroys all AWS resources to prevent billing charges.

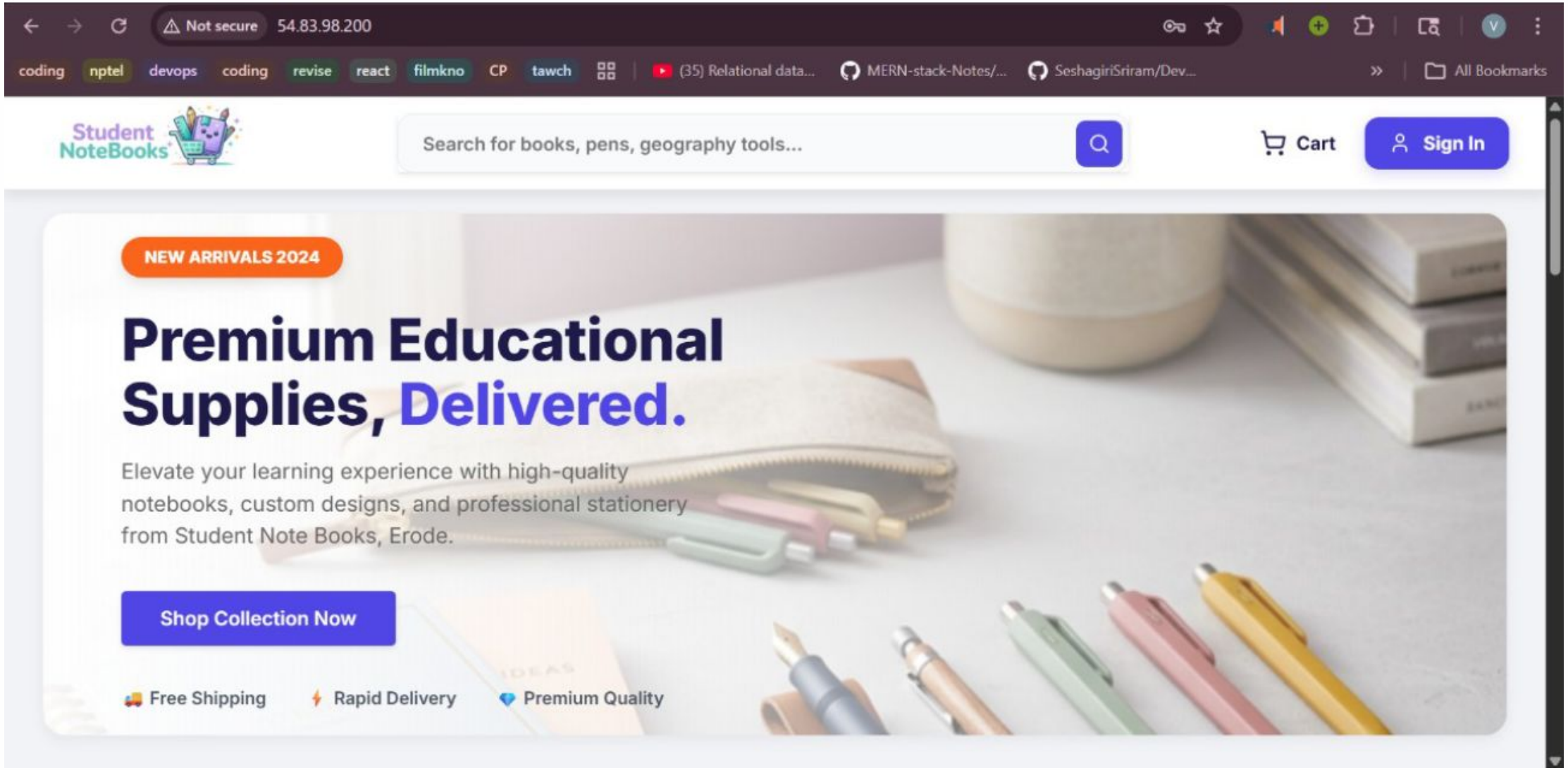
KONGU ENGINEERING COLLEGE

WEBSITE SCREENSHOT



KONGU ENGINEERING COLLEGE

WEBSITE SCREENSHOT



KONGU ENGINEERING COLLEGE

SCREENSHOT

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS powershell

```
PS C:\Users\vilva\Downloads\terraform-notes-final> terraform apply -auto-approve
>>
        id                                   = "i-001bcb48ca4cee002"
      ~ public_ip                            = "44.210.140.134" -> "174.129.138.217"
        # (39 unchanged attributes hidden)

        # (8 unchanged blocks hidden)
    }


Unless you have made equivalent changes to your configuration, or ignored the relevant attributes using ignore_changes,
the following plan may include actions to undo or respond to these changes.

---


Changes to Outputs:
  ~ backend_url  = "http://44.210.140.134:5000" -> "http://174.129.138.217:5000"
  ~ frontend_url = "http://44.210.140.134:3000" -> "http://174.129.138.217:3000"
  ~ public_ip    = "44.210.140.134" -> "174.129.138.217"
  ~ ssh_command  = "ssh -i student-notes-key.pem ubuntu@44.210.140.134" -> "ssh -i student-notes-key.pem ubuntu@174.129.13
8.217"

You can apply this plan to save these new output values to the Terraform state, without changing any real infrastructure.

Apply complete! Resources: 0 added, 0 changed, 0 destroyed.

Outputs:

backend_url = "http://174.129.138.217:5000"
frontend_url = "http://174.129.138.217:3000"
public_ip = "174.129.138.217"
ssh_command = "ssh -i student-notes-key.pem ubuntu@174.129.138.217"
PS C:\Users\vilva\Downloads\terraform-notes-final>
```

KONGU ENGINEERING COLLEGE

WORKING STATUS

Component	Status	Verified Via
Version Control (Git)	Running	GitHub (Repository is active)
CI/CD (GitHub Actions)	Success	GitHub Actions (Latest workflow: SUCCESS)
IaC (Terraform)	Running	Terraform Cloud / CLI (Infrastructure managed)
Infrastructure (AWS)	Running	AWS Console (Resources are active)
Database (MongoDB)	Running	MongoDB Atlas / EC2 (Database is online)
Backend (Node.js - PM2)	Running	PM2 (Process is online)
Frontend (React App)	Online	Live URL (App is reachable)
Monitoring (Prometheus / Grafana)	Live	Grafana Dashboard (Metrics are live)

KONGU ENGINEERING COLLEGE

RESULT

- Fully functional full-stack MERN platform deployed on AWS, supporting backend APIs (Node.js), frontend (React), and MongoDB integration with live access.

- Complete automation achieved using **Terraform** for infrastructure provisioning and **GitHub Actions CI/CD** for build, test, and deployment on every push.

- AWS EC2 instance configured with Elastic IP, security groups, and application bootstrapping. Backend managed using PM2, ensuring continuous uptime.

- Efficient deployment using minimal AWS resources (EC2 + MongoDB), with lightweight process management and optimized build setup.

KONGU ENGINEERING COLLEGE

CHALLENGES

Issues Encountered:

- Storage limitations on EC2 instance causing build failures and disk full errors.
- CI/CD pipeline failures due to dependency or build issues.
- Terraform deployment errors during provisioning (resource conflicts / misconfig).
- Port accessibility issues due to incorrect AWS Security Group rules.
- Backend process crashes when not managed properly.
- MongoDB connection issues during initial setup and seeding.

How They Were Solved:

- Increased EBS volume size and removed unnecessary files to free space.
- Debugged CI/CD logs and fixed build/configuration errors.
- Corrected Terraform scripts and re-initialized using terraform init & plan.
- Updated Security Group rules to allow required ports (HTTP, backend API).
- Used PM2 to manage and keep backend processes running reliably.
- Fixed MongoDB connection strings and verified seeding with node seeder.js.

KONGU ENGINEERING COLLEGE

CONCLUSION

- Successfully built a full-stack MERN notes platform for students
- Automated entire AWS deployment using Terraform IaC — zero manual steps
- PM2 ensures backend stays live in production
- Infrastructure is reproducible, version-controlled, and destroyable cleanly

KONGU ENGINEERING COLLEGE

KONGU ENGINEERING COLLEGE

Thank You